Kenji Manuel G. Lao (Individual)
PROJECT #9 – HOTEL CONCIERGE & BILLING SYSTEM Project Overview

1. EXECUTIVE SUMMARY

The Hotel Concierge & Billing System is a full-stack database-driven application developed as the capstone project for Database Systems 1. The system manages complete hotel operations including guest registration, room booking, service ordering, and automated billing with tax calculation.

The project successfully integrates three major technologies:

- MySQL for database management (3NF normalized schema)
- PHP for web-based CRUD operations
- Python for backend automation and billing

Key Features Implemented:

1. Complete CRUD Operations - Create, Read, Update, Delete for all 6 entities
2. Database Design (3NF) - 6 normalized tables with proper keys
3. Automated Billing System - 12% tax calculation, consolidated invoices
4. Python Integration - MySQL connector, monthly reports, invoice generation

The system meets all project requirements and demonstrates seamless integration between Database Management, Web Technologies, and Backend Logic.

2. SYSTEM OVERVIEW

2.1 Purpose

The Hotel Concierge & Billing System is designed to help hotel staff efficiently manage daily operations including:

- Registering and managing guest information
- Maintaining room inventory and availability
- Creating and modifying guest bookings
- Processing room service orders

- Generating consolidated invoices with tax calculation
- Processing guest checkouts with transaction integrity

2.2 Target Users

User Type	Responsibilities
Front Desk Staff	Manage guests, rooms, and bookings
Service Staff	Process room service orders
Billing Staff	Generate invoices and process payments
Hotel Managers	View reports and analytics

2.3 Core Features

Feature	Description
Guest Management	Add, view, edit, and delete guest records
Room Management	Manage room inventory, types, pricing, and status
Booking System	Create reservations with date validation
Service Menu	Manage hotel services (Restaurant, Spa, etc.)
Room Orders	Place service orders for active bookings
Invoicing	Generate bills with 12% tax calculation
Checkout	Process checkout with automatic room release

2.4 Technology Stack

Layer	Technology	Version	Purpose
Database	MySQL	8.0	Data storage
Backend	PHP	8.2	Server-side logic
Automation	Python	3.9	Billing engine
Frontend	HTML5/CSS3/JS	ES6	User interface
Server	Apache	2.4	Local hosting

3. DATABASE DESIGN (3NF)

3.1 Entity Relationship Diagram - Chen Notation

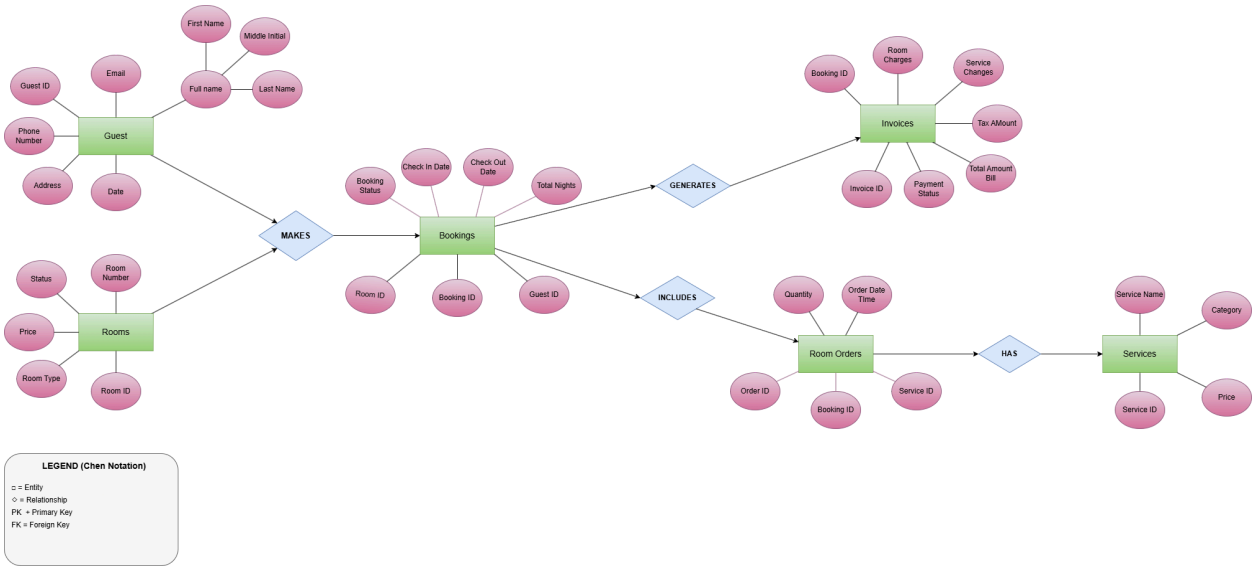

Figure 1: Chen Notation EERD

This Enhanced Entity-Relationship Diagram (EERD) uses Chen notation to represent the database structure. The diagram shows six (6) entities connected by five (5) relationships.

ENTITIES (6 total):

1. GUESTS - Stores personal information of hotel customers
2. ROOMS - Contains room inventory with types and pricing
3. BOOKINGS - Links guests to rooms with check-in/out dates
4. SERVICES - Menu of available add-on services
5. ROOM_ORDERS - Records service requests made during a stay
6. INVOICES - Generates consolidated bills for each booking

RELATIONSHIPS (5 total):

- MAKES (1:M) - One guest makes many bookings; one room has many bookings
- GENERATES (1:1) - One booking generates exactly one invoice
- INCLUDES (1:M) - One booking can include many room orders
- HAS (1:M) - One service can appear on many room orders

3.2 Entity Relationship Diagram - Crow's Foot Notation

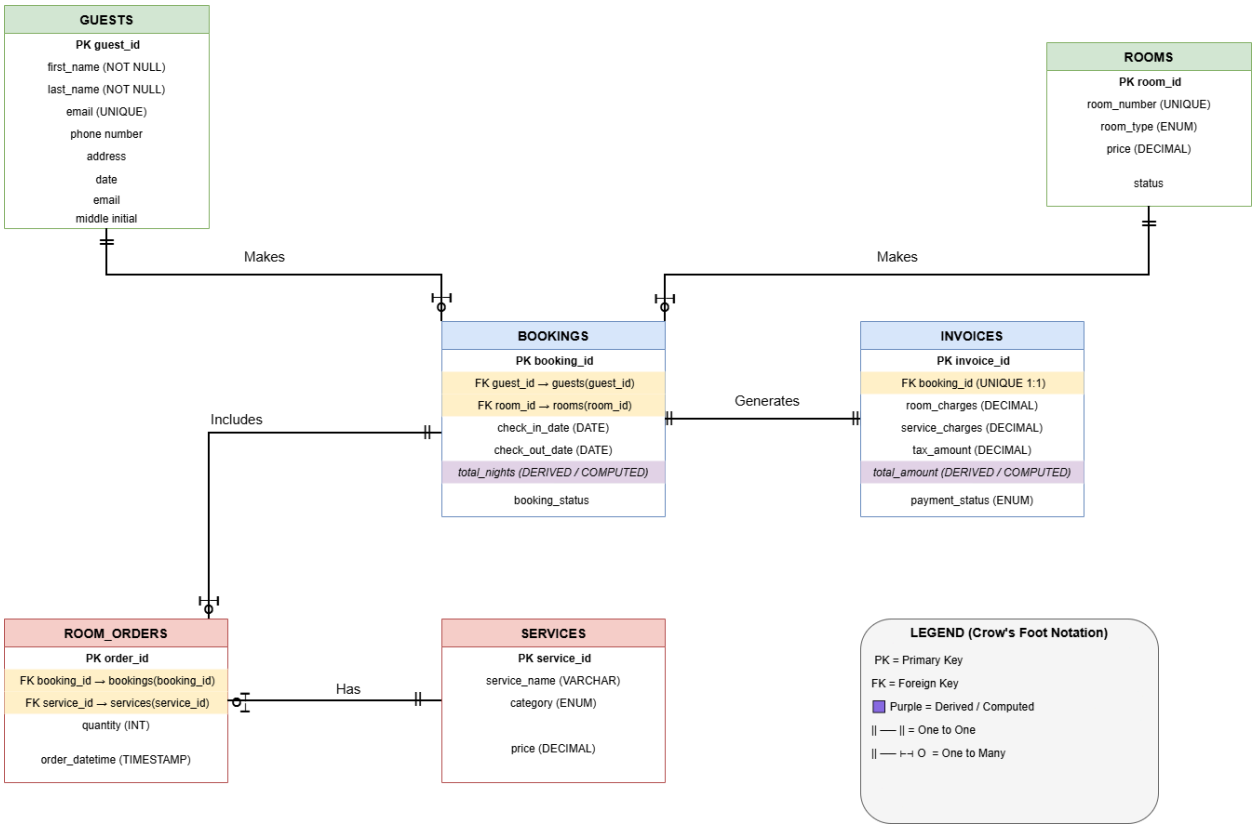

Figure 2: Crow's Foot Notation EERD

This Enhanced Entity-Relationship Diagram (EERD) uses Crow's Foot notation to represent the physical database structure. The diagram shows six (6) tables connected by five (5) relationships with cardinalities.

TABLES (6 total):

1. GUESTS - Customer master data
2. ROOMS - Room inventory
3. BOOKINGS - Reservation transactions
4. SERVICES - Price list
5. ROOM_ORDERS - Service line items
6. INVOICES - Final billing

RELATIONSHIPS (5 total):

- guests to bookings (1:M) - One guest can make many bookings

- rooms to bookings (1:M) - One room can have many bookings over time
- bookings to invoices (1:1) - One booking generates exactly one invoice
- bookings to room_orders (1:M) - One booking can include many service orders
- services to room_orders (1:M) - One service can be ordered many times

3.3 Data Dictionary

1	TABLE 1: guests				
2	Attribute	Data Type	Constraints	Primary Key	Foreign Key	Description
3	guest_id	INT	PRIMARY KEY, AUTO_INCREMENT	YES	NO	Unique identifier for each guest
4	first_name	VARCHAR	NOT NULL	NO	NO	Guest's first name
5	last_name	VARCHAR	NOT NULL	NO	NO	Guest's last name
6	email	VARCHAR	UNIQUE, NOT NULL	NO	NO	Guest's email address
7	phone	VARCHAR	None	NO	NO	Contact phone number
8	address	TEXT	None	NO	NO	Complete residential address
9	created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	NO	NO	Registration date and time
10						
11	TABLE 2: rooms					
12	Attribute	Data Type	Constraints	Primary Key	Foreign Key	Description
13	room_id	INT	PRIMARY KEY, AUTO_INCREMENT	YES	NO	Unique room identifier
14	room_number	VARCHAR	UNIQUE, NOT NULL	NO	NO	Physical room number/name
15	room_type	ENUM	NOT NULL	NO	NO	Category of room (Standard, Deluxe, Suite, Presidential)
16	price_per_night	DECIMAL	CHECK > 0, NOT NULL	NO	NO	Cost per night in local currency
17	status	ENUM	NOT NULL	NO	NO	Current room availability (Available, Occupied, Maintenance, Reserved)
18						
19	TABLE 3: bookings					
20	Attribute	Data Type	Constraints	Primary Key	Foreign Key	Description
21	booking_id	INT	PRIMARY KEY, AUTO_INCREMENT	YES	NO	Unique booking reference
22	guest_id	INT	FOREIGN KEY, NOT NULL	NO	YES (guests.g	Guest making the booking
23	room_id	INT	FOREIGN KEY, NOT NULL	NO	YES (rooms.r	Room being booked
24	check_in_date	DATE	NOT NULL	NO	NO	Arrival date
25	check_out_date	DATE	NOT NULL	NO	NO	Departure date
26	total_nights	INT	GENERATED, STORED (DERIVED)	NO	NO	Number of nights (auto-calculated)
27	booking_status	ENUM	NOT NULL	NO	NO	Current booking state (Active, CheckedOut, Cancelled)
28						
29	TABLE 4: services					
30	Attribute	Data Type	Constraints	Primary Key	Foreign Key	Description
31	service_id	INT	PRIMARY KEY, AUTO_INCREMENT	YES	NO	Unique service item ID
32	service_name	VARCHAR	NOT NULL	NO	NO	Name of service/product
33	category	ENUM	NOT NULL	NO	NO	Service department category (Restaurant, Spa, MiniBar, Laundry, Other)
34	price	DECIMAL	CHECK > 0, NOT NULL	NO	NO	Cost per unit/item
35						

36	TABLE 5: room_orders				
37	Attribute	Data Type	Constraints	Primary Key	Foreign Key	Description
38	order_id	INT	PRIMARY KEY, AUTO_INCREMENT	YES	NO	Unique order reference
39	booking_id	INT	FOREIGN KEY, NOT NULL	NO	YES (booking	Associated booking
40	service_id	INT	FOREIGN KEY, NOT NULL	NO	YES (services	Service ordered
41	quantity	INT	CHECK > 0, NOT NULL	NO	NO	Number of units ordered
42	order_datetime	DATETIME	DEFAULT CURRENT_TIMESTAMP	NO	NO	When order was placed
43						
44	TABLE 6: invoices					
45	Attribute	Data Type	Constraints	Primary Key	Foreign Key	Description
46	invoice_id	INT	PRIMARY KEY, AUTO_INCREMENT	YES	NO	Unique invoice number
47	booking_id	INT	FOREIGN KEY, UNIQUE, NOT NULL	NO	YES (booking	Associated booking (one-to-one)
48	room_charges	DECIMAL	DEFAULT 0	NO	NO	Total room cost (nights × price)
49	service_charges	DECIMAL	DEFAULT 0	NO	NO	Total of all service orders
50	tax_amount	DECIMAL	DEFAULT 0	NO	NO	Tax calculated (12% of subtotal)
51	total_amount	DECIMAL	GENERATED, STORED (DERIVED)	NO	NO	Grand total (auto-calculated)
52	payment_status	ENUM	NOT NULL	NO	NO	Payment tracking (Unpaid, Paid, PartiallyPaid)
53	invoice_date	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	NO	NO	Bill generation date
54						

Database Sample Data

TABLE 1: guests (8 records)

guest_id	first_name	last_name	email	phone	address	created_at
1	John	Smith	john.smith@email.com	555-0101	123 Main St, New York, NY 10001	2026-04-10 00:00:00
2	Maria	Garcia	maria.garcia@email.com	555-0102	456 Oak Ave, Los Angeles, CA 90001	2026-04-10 00:00:00
3	David	Lee	david.lee@email.com	555-0103	789 Pine Rd, Chicago, IL 60601	2026-04-10 00:00:00
4	Sarah	Johnson	sarah.j@email.com	555-0104	321 Elm St, Houston, TX 77001	2026-04-10 00:00:00
5	Robert	Brown	robert.brown@email.com	555-0105	654 Cedar Ln, Phoenix, AZ 85001	2026-04-10 00:00:00
6	Lisa	Wilson	lisa.wilson@email.com	555-0106	987 Birch Dr, Philadelphia, PA 19101	2026-04-10 00:00:00
7	James	Martinez	james.m@email.com	555-0107	147 Maple Way, San Antonio, TX 78201	2026-04-10 00:00:00
8	Emily	Taylor	emily.taylor@email.com	555-0108	258 Walnut St, San Diego, CA 92101	2026-04-10 00:00:00

TABLE 2: rooms (8 records)

room_id	room_number	room_type	price_per_night	status
1	101	Standard	120.00	Available
2	102	Standard	120.00	Occupied
3	103	Deluxe	180.00	Available
4	104	Deluxe	180.00	Reserved
5	201	Suite	250.00	Available
6	202	Suite	250.00	Occupied
7	301	Presidential	500.00	Available
8	302	Presidential	500.00	Maintenance

TABLE 3: services (10 records)

service_id	service_name	category	price
1	Club Sandwich	Restaurant	12.50
2	Caesar Salad	Restaurant	10.00
3	Grilled Cheese	Restaurant	8.50
4	Massage (60min)	Spa	85.00
5	Facial Treatment	Spa	65.00
6	Minibar - Soda	MiniBar	4.00
7	Minibar - Beer	MiniBar	6.00
8	Minibar - Wine	MiniBar	12.00
9	Laundry (Shirt)	Laundry	5.50
10	Laundry (Suit)	Laundry	15.00

TABLE 4: bookings (6 records)

booking_id	guest_id	room_id	check_in_date	check_out_date	total_nights	booking_status
1	1	1	2026-04-10	2026-04-15	5	Active
2	2	3	2026-04-11	2026-04-14	3	Active
3	3	5	2026-04-12	2026-04-18	6	Active
4	4	4	2026-04-09	2026-04-13	4	CheckedOut
5	5	6	2026-04-10	2026-04-20	10	Active
6	6	2	2026-04-15	2026-04-18	3	Reserved

TABLE 5: room_orders (8 records)

order_id	booking_id	service_id	quantity	order_datetime
1	1	1	2	2026-04-10 14:30:00
2	1	6	3	2026-04-10 14:30:00
3	2	4	1	2026-04-11 09:15:00
4	2	9	2	2026-04-11 09:15:00
5	3	2	1	2026-04-12 18:00:00
6	3	7	2	2026-04-12 18:00:00
7	5	4	1	2026-04-10 20:30:00
8	5	5	1	2026-04-10 20:30:00

TABLE 6: invoices (1 record)

invoice_id	booking_id	room_charges	service_charges	tax_amount	total_amount	payment_status	invoice_date
1	4	480.00	25.00	60.60	565.60	Paid	2026-04-13 00:00:00

Table Name	Number of Records	Primary Key	Foreign Keys
guests	8	guest_id	None
rooms	8	room_id	None
services	10	service_id	None
bookings	6	booking_id	guest_id, room_id
room_orders	8	order_id	booking_id, service_id
invoices	1	invoice_id	booking_id

3.4 Normalization Explanation

The database is designed in Third Normal Form (3NF) to eliminate data redundancy and ensure data integrity.

First Normal Form (1NF) - No Repeating Groups

Each table has atomic values with no repeating groups. For example, the room_orders table stores each service as a separate row rather than comma-separated values.

Second Normal Form (2NF) - No Partial Dependencies

All non-key attributes depend on the entire primary key. The bookings table has a composite key (booking_id) and all attributes depend on it.

Third Normal Form (3NF) - No Transitive Dependencies

All attributes depend only on the primary key, not on other non-key attributes. For example, total_amount in invoices is derived from room_charges + service_charges + tax, not stored directly.

Why 3NF?

Benefit	Description
Eliminates Redundancy	Same data not repeated across rows
Prevents Update Anomalies	Changing data in one place updates everywhere
Prevents Insertion Anomalies	Can add new data without affecting others
Prevents Deletion Anomalies	Deleting data doesn't lose other information

Summary of Normalization:

Table	1NF	2NF	3NF	Status
guests	Yes	Yes	Yes	3NF
rooms	Yes	Yes	Yes	3NF
bookings	Yes	Yes	Yes	3NF
services	Yes	Yes	Yes	3NF
room_orders	Yes	Yes	Yes	3NF
invoices	Yes	Yes	Yes	3NF

4. SYSTEM ARCHITECTURE

The system follows a three-tier architecture:

Presentation Tier (HTML, CSS, JavaScript, PHP)

- Web Browser → User Interface → Forms → Tables → Dashboard

Application Tier (PHP & Python)

- PHP: CRUD Operations, Form Processing, Session Management
- Python: Billing Engine, Report Generation, Email Automation

Data Tier (MySQL)

- Tables: guests, rooms, bookings, services, room_orders, invoices
- Views: consolidated_charges
- Procedures: CheckoutGuest

Data Flow:

1. User interacts with web interface (PHP forms)
2. PHP validates input and sends SQL queries to MySQL
3. MySQL processes queries and returns results
4. PHP displays results in HTML tables
5. Python scripts can also access MySQL for reporting
6. Invoices are generated using complex JOIN queries

5. PHP CRUD OPERATIONS

5.1 Create Operation (Adding Data)

How it works:

When a user fills out a form and clicks submit, PHP takes the information and inserts it into the database as a new row.

Example - Adding a Guest:

The user enters first name, last name, and email. PHP receives this data and runs an INSERT SQL command. The database adds a new row to the guests table. A success message confirms the guest was added.

Validation: JavaScript checks that required fields are not empty before sending to PHP.

5.2 Read Operation (Viewing Data)

How it works:

PHP asks the database for all records in a table. The database returns the data. PHP displays it in an HTML table on the webpage.

Example - Viewing Guests:

PHP runs a SELECT command to get all guests. The database returns every row from the guests table. PHP loops through each row and creates a table row for each guest. The user sees a complete list of all guests.

Complex Reads (Joins):

For bookings, PHP needs information from multiple tables. It uses a JOIN command to combine guests (for names), rooms (for room numbers), and bookings (for dates). This creates a complete view of each booking.

5.3 Update Operation (Editing Data)

How it works:

When a user clicks "Edit" next to a record, PHP fetches the current data and fills it into a form. The user changes the information and submits. PHP runs an UPDATE command to save the changes.

Example - Editing a Guest:

PHP gets the guest ID from the URL. It fetches that guest's current information. The form is pre-filled with the existing data. The user changes the phone number and submits. PHP updates only that specific guest's record.

5.4 Delete Operation (Removing Data)

How it works:

When a user clicks "Delete," JavaScript asks for confirmation. If confirmed, PHP runs a DELETE command to remove that specific record.

Safety Checks:

Before deleting a service, PHP checks if any orders exist for that service. If orders exist, the deletion is blocked to prevent data inconsistency.

Example - Deleting a Guest:

PHP gets the guest ID from the URL. It runs a DELETE command where guest_id matches. The guest is removed from the database. The page refreshes and the guest no longer appears in the list.

6. PYTHON INTEGRATION

6.1 Python-MySQL Connection

How it works:

Python uses a library called mysql-connector-python to connect to the same MySQL database that PHP uses. This allows Python to read data and perform calculations.

What Python can do:

- Read guest and booking information
- Calculate invoice totals with tax
- Generate monthly revenue reports
- Export invoices as HTML files

Connection Process:

Python provides the database credentials (host, database name, username, password). It establishes a connection. Python can now run SELECT queries just like PHP. The connection is closed when finished.

6.2 Billing Engine Logic

How the billing engine works:

1. Get Booking Details - Python fetches guest name, room number, dates, and price per night
2. Calculate Room Charges - Multiply nights by price per night
3. Get Service Orders - Fetch all services ordered for this booking
4. Calculate Service Charges - Sum up (quantity × price) for each service
5. Calculate Tax - Add room charges and service charges, multiply by 0.12 (12%)
6. Calculate Grand Total - Add room charges + service charges + tax

Example Calculation:

- 5 nights at $120 = $600 room charges
- 2 Club Sandwiches ($25) + 3 Sodas ($12) = $37 service charges
- Subtotal = $637

- Tax (12%) = $76.44
- Grand Total = $713.44

6.3 Monthly Reporting Logic

How monthly reports work:

1. Python asks the database for all completed bookings in a specific month
2. The database returns total bookings, total revenue, and average bill
3. Python displays this information in a formatted report

What the report shows:

- How many guests checked out this month
- Total money collected from all invoices
- Average amount spent per guest
- Breakdown of room revenue vs service revenue

Output Example:

Period: April 2026
Total Bookings: 5
Total Revenue: $2,450.00
Average Bill: $490.00

7. TRANSACTION LOGIC

What is a Transaction?

A transaction is a group of database operations that must all succeed together. If any operation fails, all changes are undone (rolled back).

The Checkout Process

When a guest checks out, three things must happen:

1. Generate Invoice - Calculate room charges, service charges, and tax, then save to invoices table
2. Update Booking Status - Change booking status from "Active" to "CheckedOut"
3. Release the Room - Change room status from "Occupied" to "Available"

Why Transactions Matter

Imagine if the system crashed after step 2 but before step 3:

- The invoice would be generated
- The booking would show as checked out
- BUT the room would still show as occupied

The hotel would think the room is still in use when it's actually empty.

How Our Transaction Works

Normal Flow (Everything succeeds):

1. Start Transaction
2. Generate Invoice → Success
3. Update Booking Status → Success
4. Release Room → Success
5. Commit (Save all changes permanently)

Error Flow (Something fails):

1. Start Transaction
2. Generate Invoice → Success
3. Update Booking Status → Success
4. Release Room → FAILED (database error)
5. ROLLBACK (Undo steps 1, 2, and 3)
6. Database returns to original state

The Rollback Safety Net

The system has an "exit handler" that watches for any error. If an error occurs at any point, the handler triggers a ROLLBACK command. This undoes every change made

during the transaction. The database returns to exactly how it was before the checkout started.

This ensures data integrity - No partial transactions are ever saved.

8. USER MANUAL

8.1 How to Add a Guest

1. From the Dashboard, click "Add New Guest"
2. Enter first name, last name, and email (required)
3. Optional: Enter phone number and address
4. Click "Save Guest"
5. Click "View All Guests" to see the new guest in the list

8.2 How to Add a Room

1. From the Dashboard, click "Add New Room"
2. Enter room number (e.g., "101")
3. Select room type (Standard, Deluxe, Suite, or Presidential)
4. Enter price per night (e.g., "120.00")
5. Select status (Available, Occupied, Maintenance, or Reserved)
6. Click "Save Room"

8.3 How to Create a Booking

1. Click "New Booking"
2. Select a guest from the dropdown menu
3. Select an available room from the dropdown
4. Choose check-in date (cannot be in the past)
5. Choose check-out date (must be after check-in)
6. Click "Create Booking"

What happens automatically:

- Total nights are calculated

- Room status changes to "Occupied"
- Booking appears in the Active Bookings list

8.4 How to Place a Room Order

1. Click "Place Order"
2. Select an active booking from the dropdown
3. Select a service from the menu
4. Enter quantity (minimum 1)
5. Click "Place Order"
6. The order appears in "Recent Orders" table

8.5 How to Generate an Invoice

1. Click "View Invoices"
2. To generate a new invoice:
    - Select an active booking
    - Click "Generate Invoice & Checkout"
3. To view existing invoices:
    - Click "View Invoice" next to any invoice
4. To update payment status:
    - Select Paid, Unpaid, or Partially Paid
    - Click "Update Payment Status"
5. To print: Click "Print Invoice"

8.6 How to Run Python Reports

1. Open Command Prompt
2. Navigate to the python folder: cd C:\xampp\htdocs\hotel_system\python
3. Run: python billing_engine.py
4. Enter a Booking ID when prompted
5. View the invoice in the console
6. Type "y" to save as HTML file

9. TESTING & VALIDATION

Test Case	Expected Result	Actual Result	Status
Add Guest	Guest appears in table	Guest appears	PASS
View Guests	Table shows all guests	Table shows all	PASS
Edit Guest	Changes saved	Changes saved	PASS
Delete Guest	Guest removed	Guest removed	PASS
Add Room	Room added to inventory	Room added	PASS
Create Booking	Booking created, room occupied	Success	PASS
Place Order	Order added to list	Order added	PASS
Generate Invoice	Invoice with tax calculated	Tax calculated	PASS

9.2 Form Validation Test Results

Test	Input	Expected	Actual	Status
Empty First Name	""	Alert message	Alert shown	PASS
Invalid Email	"test"	Alert message	Alert shown	PASS
Negative Price	-10	Alert message	Alert shown	PASS
Past Check-In	2020-01-01	Alert message	Alert shown	PASS
Check-Out before Check-In	Earlier date	Alert message	Alert shown	PASS

9.3 Transaction Test Results

Test	Action	Expected	Actual	Status
Normal Checkout	Process booking	Invoice created, room released	Success	PASS
Duplicate Checkout	Process same booking twice	Invoice updates, no duplicate	Success	PASS
Invalid Booking	Process fake booking	Error message	Error shown	PASS

9.4 Python Integration Test Results

Test			
Datab			
Get B			
Calculate Invoice	Grand total with tax	Tax calculated	PASS
Monthly Report	Revenue numbers	Aggregates data	PASS

10. CONCLUSION

The Hotel Concierge & Billing System has successfully met all project requirements.

Requirements Met:

Requirement	Status	Evidence
3NF Database Design	COMPLETE	6 normalized tables with proper keys
Complex SQL Joins	COMPLETE	Consolidated view combining multiple tables
Transaction Logic	COMPLETE	Checkout procedure with rollback
PHP CRUD Operations	COMPLETE	Create, Read, Update, Delete for all tables
Python Integration	COMPLETE	Billing engine, reports, HTML export
CSS Styling	COMPLETE	Professional dark theme
JavaScript Validation	COMPLETE	Form validation, delete confirmation

Key Achievements:

1. Seamless integration between MySQL, PHP, and Python working together
2. Automated invoice generation with 12% tax calculation
3. Transaction-safe checkout that prevents data corruption
4. Professional, responsive user interface
5. Comprehensive reporting through Python backend